# TASK 9 – Implement and Test the Calculator

## Implementation

The implementation below uses Decimal values created from text, validates quantity and fat range, calculates the signed 0.1% steps, and rounds the rate and payment to two decimal places. The assignment does not provide an SNF limit, so the code checks that SNF is a finite number but does not invent an SNF range.

## Complete executable program

```
from decimal import Decimal, InvalidOperation, ROUND_HALF_UP

BASE_RATE = Decimal("30.00")
REFERENCE_FAT = Decimal("3.5")
REFERENCE_SNF = Decimal("8.5")
FAT_STEP_VALUE = Decimal("0.40")
SNF_STEP_VALUE = Decimal("0.30")
STEP = Decimal("0.1")
MIN_FAT = Decimal("2.0")
MAX_FAT = Decimal("7.0")
CENT = Decimal("0.01")

def money(value):
    return value.quantize(CENT, rounding=ROUND_HALF_UP)

def calculate_rate(fat, snf):
    fat_steps = ((fat - REFERENCE_FAT) / STEP).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
    snf_steps = ((snf - REFERENCE_SNF) / STEP).quantize(Decimal("1"), rounding=ROUND_HALF_UP)
    return money(BASE_RATE + fat_steps * FAT_STEP_VALUE + snf_steps * SNF_STEP_VALUE)

def calculate_payment(litres, fat, snf):
    if not litres.is_finite() or litres <= 0:
        raise ValueError("Litres must be a positive finite number")
    if not fat.is_finite() or not MIN_FAT <= fat <= MAX_FAT:
        raise ValueError("Fat must be between 2.0 and 7.0 percent")
    if not snf.is_finite():
        raise ValueError("SNF must be finite")
    return money(litres * calculate_rate(fat, snf))

def run_tests():
    cases = [
        ("standard", "10", "3.5", "8.5", "300.00"),
        ("above standard", "8.5", "4.2", "8.7", "283.90"),
        ("below standard", "12", "3.1", "8.3", "333.60"),
        ("minimum fat", "1", "2.0", "8.5", "24.00"),
        ("maximum fat", "1", "7.0", "8.5", "44.00"),
    ]
    for label, litres, fat, snf, expected in cases:
        actual = calculate_payment(Decimal(litres), Decimal(fat), Decimal(snf))
        assert actual == Decimal(expected), (label, actual, expected)
        print(label, "PASS", "expected=", expected, "actual=", actual)
    for label, values in [("negative quantity", ("-1", "3.5", "8.5")),
                          ("fat too low", ("1", "1.9", "8.5")),
                          ("fat too high", ("1", "7.1", "8.5"))]:
        try:
            calculate_payment(*(Decimal(x) for x in values))
        except ValueError:
            print(label, "PASS (rejected)")
        else:
            raise AssertionError(label + " should have been rejected")
    try:
        Decimal("eight")
    except InvalidOperation:
        print("text input PASS (rejected during conversion)")

if __name__ == "__main__":
    run_tests()
```

## Test table

Expected/actual cases: standard = Rs 300.00; above standard = Rs 283.90; below standard = Rs 333.60; minimum fat 2.0% at 1 litre = Rs 24.00; maximum fat 7.0% at 1 litre = Rs 44.00. Negative litres, fat 1.9%, fat 7.1%, and non-numeric text must be rejected. The program asserts each expected amount and prints PASS only when the actual result agrees.

## Testing approach and complexity

Manual testing requires a person to re-enter each case after every change and may omit cases. An assert-based test function runs the same cases consistently and quickly after each edit. If there are n test cases, a complete run is O(n) time and O(1) extra space apart from the fixed test data. Automated tests are repeatable and reduce regression risk; manual tests remain useful for checking the interactive user experience.